

Module 6: Strings in Java

Java Laboratory – Scenario-Based Problem Statements

Module Overview

Strings are among the most frequently used data types in Java applications. Almost every software application—from banking systems and e-commerce platforms to healthcare portals and social media applications—processes textual information. This module introduces students to Java's string handling capabilities using the **String**, **StringBuilder**, and **StringBuffer** classes.

The laboratory exercises focus on solving real-world problems involving data validation, text processing, report generation, search operations, and formatting. Students will also learn the importance of immutable and mutable strings and their applications in enterprise software development.

Total Practice Problems: 25

Difficulty Distribution

- **Beginner:** 8 Problems
 - **Intermediate:** 10 Problems
 - **Advanced:** 7 Problems
-

Learning Objectives

After completing this module, students will be able to:

- Create and manipulate strings.
 - Apply built-in String methods effectively.
 - Compare strings correctly.
 - Validate user inputs.
 - Perform searching and pattern matching.
 - Use StringBuilder and StringBuffer efficiently.
 - Develop text-processing applications.
 - Build real-world applications involving string manipulation.
-

Concepts Covered

- String Class
 - String Objects
 - String Methods
 - String Comparison
 - StringBuilder
 - StringBuffer
 - String Tokenization
 - String Formatting
 - Pattern Matching
 - Text Processing
-

Beginner Level Problems

Problem 1: Student Registration Validation

Scenario

A university admission portal validates student details before registration.

Problem Statement

Develop a Java application to:

- Accept the student's full name.
 - Remove leading and trailing spaces.
 - Convert the name to Proper Case or Uppercase.
 - Display the total number of characters.
 - Display the initials of the student.
-

Problem 2: Email Address Validator

Scenario

A company allows only valid email addresses during employee registration.

Problem Statement

Write a Java program that accepts an email address and verifies whether it:

- Contains '@'
- Contains '.'
- Does not begin or end with special characters
- Has a valid domain name

Display whether the email is valid or invalid.

Problem 3: Password Strength Checker

Scenario

A banking application requires strong passwords.

Problem Statement

Accept a password and determine whether it satisfies the following conditions:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character

Display the password strength.

Problem 4: Employee ID Formatter

Scenario

An organization wants employee IDs in a standard format.

Problem Statement

Accept an employee name and employee number.

Generate an Employee ID using the following format:

EMP-<FirstThreeLetters>-<EmployeeNumber>

Problem 5: Customer Feedback Analyzer

Scenario

An online shopping platform stores customer feedback.

Problem Statement

Accept customer feedback and display:

- Number of words
 - Number of characters
 - Number of vowels
 - Number of consonants
-

Problem 6: Mobile Number Formatter

Scenario

A telecom company stores customer phone numbers.

Problem Statement

Accept a mobile number.

Validate its length and format it as:

+91 XXXXX XXXXX

(or another suitable country format).

Problem 7: Username Generator

Scenario

A company automatically generates usernames.

Problem Statement

Accept:

- First Name
- Last Name

Generate a username using an appropriate naming convention.

Example:

`john.smith`

Problem 8: Reverse Customer Name

Scenario

A data migration system needs names in reverse order.

Problem Statement

Accept a customer's full name and display:

- Original Name
 - Reversed String
 - Individual words in reverse order
-

Intermediate Level Problems

Problem 9: Product Search

Scenario

An e-commerce website allows users to search products.

Problem Statement

Store product names.

Allow users to search for a product using partial text.

Display matching product names.

Problem 10: Resume Keyword Analyzer

Scenario

A recruitment portal scans resumes for important technical skills.

Problem Statement

Accept resume text and count occurrences of keywords such as:

- Java
- Spring Boot
- SQL
- Docker

Display the keyword frequency.

Problem 11: Chat Message Formatter

Scenario

A messaging application formats user messages before display.

Problem Statement

Accept a message and:

- Remove extra spaces
- Capitalize the first letter of each sentence
- Replace inappropriate words with "***"

Display the formatted message.

Problem 12: Library Book Search

Scenario

A digital library allows searching books.

Problem Statement

Store book titles.

Allow users to search books using:

- Prefix
- Suffix
- Partial title

Display all matching books.

Problem 13: Website URL Analyzer

Scenario

A browser analyzes website addresses.

Problem Statement

Accept a URL and display:

- Protocol
 - Domain Name
 - Top-Level Domain (TLD)
 - Path (if available)
-

Problem 14: Student Roll Number Validator

Scenario

A university has a predefined roll number format.

Problem Statement

Accept a roll number and validate whether it follows the university format.

Display an appropriate message.

Problem 15: Invoice Generator

Scenario

A retail company generates customer invoices.

Problem Statement

Accept customer and product information.

Generate a neatly formatted invoice using formatted strings.

Problem 16: Search and Replace Utility

Scenario

A publishing company edits large documents.

Problem Statement

Accept a paragraph.

Replace every occurrence of a specified word with another word.

Display the updated paragraph.

Problem 17: StringBuilder Performance Demonstration

Scenario

A software company processes large amounts of text.

Problem Statement

Develop a program that repeatedly appends text using:

- String
- StringBuilder

Observe and compare the execution time.

Discuss which approach is more efficient and why.

Problem 18: Bank Account Number Masking

Scenario

An online banking application hides sensitive account information.

Problem Statement

Accept an account number.

Display only the last four digits.

Mask the remaining digits using '*'.

Example:

*****5678

Advanced Level Problems

Problem 19: Hospital Patient Record Search

Scenario

A hospital stores patient names.

Problem Statement

Allow users to search patient names irrespective of uppercase or lowercase characters.

Implement case-insensitive searching.

Problem 20: Social Media Hashtag Analyzer

Scenario

A social media platform analyzes trending hashtags.

Problem Statement

Accept a paragraph containing hashtags.

Display:

- Total hashtags
 - Unique hashtags
 - Most frequently used hashtag
-

Problem 21: College Notice Formatter

Scenario

A university generates official notices.

Problem Statement

Accept notice details and generate a professionally formatted notice using `StringBuilder`.

Problem 22: Log File Analyzer

Scenario

A server generates thousands of log messages every day.

Problem Statement

Accept multiple log entries and determine:

- Number of ERROR messages
- Number of WARNING messages
- Number of INFO messages

Generate a summary report.

Problem 23: DNA Sequence Analyzer

Scenario

A biotechnology laboratory stores DNA sequences.

Problem Statement

Accept a DNA sequence and determine:

- Number of Adenine (A)
- Cytosine (C)
- Guanine (G)
- Thymine (T)

Display the nucleotide frequency report.

Problem 24: Online Examination Answer Evaluator

Scenario

An online examination system compares student answers with the correct answer key.

Problem Statement

Accept:

- Correct answer string
- Student response string

Determine:

- Correct answers
 - Incorrect answers
 - Score Percentage
-

Problem 25: Text Compression Analyzer

Scenario

A cloud storage company wants to identify repeated words before compression.

Problem Statement

Accept a paragraph and generate a frequency table showing the occurrence of each unique word.

Display the words in descending order of frequency.

Challenge Problems

1. Develop a **Spam Email Detection System** based on suspicious keywords.
 2. Create a **Chat Profanity Filter** that replaces offensive words with appropriate symbols.
 3. Build a **Plagiarism Checker** by comparing two text documents and identifying common sentences.
 4. Design a **URL Shortener Simulator** that generates unique short links from long URLs.
 5. Create a **Student ID Generator** based on department, year of admission, and student name.
 6. Develop a **Text Encryption and Decryption Utility** using a simple substitution or Caesar cipher.
 7. Build a **Markdown to Plain Text Converter** that removes Markdown formatting.
 8. Implement a **Command-Line Search Utility** that searches multiple text files for a keyword and highlights the matches.
-

Real-World Applications

Students will understand how strings are used in:

- Banking applications
 - Hospital Management Systems
 - Search engines
 - Chat and messaging applications
 - Email systems
 - Resume screening software
 - Digital libraries
 - E-commerce platforms
 - Government portals
 - Enterprise reporting systems
-

Common Mistakes

Students should avoid:

- Comparing strings using `==` instead of `equals()` or `equalsIgnoreCase()`.
 - Ignoring whitespace during input validation.
 - Not handling empty or null strings appropriately.
 - Assuming strings are mutable.
 - Using `String` repeatedly for large concatenation operations instead of `StringBuilder`.
 - Overlooking case sensitivity in search operations.
 - Misusing string indexes, leading to `StringIndexOutOfBoundsException`.
 - Writing inefficient loops for text processing.
-

Viva Questions

1. Why are `String` objects immutable in Java?
 2. Differentiate between `String`, `StringBuilder`, and `StringBuffer`.
 3. Why should `equals()` be used instead of `==` for string comparison?
 4. Explain the concept of the String Pool.
 5. What are the advantages of immutable strings?
 6. When should `StringBuilder` be preferred over `String`?
 7. Differentiate between `equals()` and `equalsIgnoreCase()`.
 8. What are some commonly used `String` methods for searching and manipulation?
 9. How does string processing play a role in enterprise applications?
 10. What are the performance implications of repeated string concatenation?
-

Expected Learning Outcomes

After completing this module, students will be able to:

- Manipulate and process textual data using Java's string classes.
- Validate and format user inputs for real-world applications.
- Choose the appropriate string class (`String`, `StringBuilder`, or `StringBuffer`) based on performance and thread-safety requirements.
- Develop practical text-processing utilities used in enterprise software.
- Write efficient, readable, and maintainable programs involving string operations.

Special Notes

- Encourage students to analyze a problem before selecting the appropriate string class.
- Ask students to compare implementations using `String`, `StringBuilder`, and `StringBuffer` where applicable.
- Introduce discussions on immutability, memory efficiency, and thread safety.
- Relate the exercises to real-world software such as login systems, search engines, document processing tools, chat applications, and banking portals to reinforce the practical importance of string manipulation.